Online Code Judge: Task Document

Duration: 8 weeks | **Check-ins:** weekly, 20-30 minutes with your mentor | **Type:** standalone full-stack project (your own repo, your own machine, fabricated data only)

You will build a small version of the platforms people use to practise coding: read a problem, write code in the browser, press Submit, and get a **verdict** (Accepted, Wrong Answer, Time Limit Exceeded…) back within seconds, with a live leaderboard.

The interesting part is **what happens between Submit and the verdict**: a queue, a worker that runs *untrusted* code inside a locked-down container, live updates pushed to the browser, rate limits, and a leaderboard that updates instantly. Most of these technologies will be new to you. That is the point: you are expected to read, experiment, break things and learn, and your mentor will guide you.

This document has five parts:

1. **Requirements Document**: what the platform does.
2. **Safety Rules & Architecture**: the non-negotiable safety rules for running untrusted code, and how the services fit together.
3. **Expected Outcomes & Acceptance Criteria**: Given / When / Then checks used to review your work.
4. **Tech Stack Requirements**.
5. **Submission Details**: weekly plan, tiers, deliverables and scoring.

Ground rules (read first)

- You will **not** be given access to any company code, servers, databases or credentials. This is a standalone project.
- All data is **fabricated by you** (problems, users, test cases). Never use real personal data. Never commit secrets: provide a `.env.example` only.
- **Read Part 2 (Safety Rules) before writing any code.** This project runs code that it does not trust. Breaking a safety rule is the one thing that can fail the project, no matter how good the rest is.
- **Ask before you build** if anything is unclear. Write every assumption in `docs/known-issues.md` or `docs/changelog.md`.
- Every week you must **demo something that runs**, even if small. Hiding a problem until the check-in costs you marks.

PART 1: REQUIREMENTS DOCUMENT

Purpose: Define the product, roles and functional requirements of the Online Code Judge.

Out of scope: Real payments, cloud deployment, public hosting, plagiarism detection (stretch), languages other than JavaScript (stretch), real email providers.

Audience: Product, QA and Developers.

1) Objectives

- Let **users** register, browse problems, write and submit code, see live verdicts, view their history, and compete on a leaderboard.
- Let **problem setters** create problems with sample and hidden test cases.
- Let **admins** run timed contests and watch the health of the judge.
- **Run submitted code safely**, with strict time, memory and output limits, no network and no access to the host.
- Behave correctly under load and failure: no lost submissions, no duplicated verdicts, no stuck "judging" states, no leftover containers.
- Be fully runnable on one laptop (the safety rules decide *where* the worker runs).

2) Roles (RBAC)

Role	Can do
User	Register, log in, browse and search problems, submit code, view own submissions, view leaderboards, join contests
Problem setter	Everything a user can, plus create and edit problems and test cases
Admin	Everything above, plus create contests, view judge health, view and re-judge any submission, kill a stuck run

3) Core Definitions

- **Problem:** a statement, an input / output format, limits (time and memory), **sample** test cases (visible) and **hidden** test cases (never shown).
- **Submission:** one run of one user's code against one problem.
- **Test case:** an input file and an expected output file.
- **Verdict:** the final result of a submission:
  - `ACCEPTED` : every test passed.
  - `WRONG_ANSWER` : output differed on some test.
  - `TIME_LIMIT_EXCEEDED` : ran longer than the time limit.
  - `MEMORY_LIMIT_EXCEEDED` : used more memory than the limit.
  - `RUNTIME_ERROR` : crashed (exception, non-zero exit).
  - `COMPILATION_ERROR` : code could not be parsed (syntax error).
  - `OUTPUT_LIMIT_EXCEEDED` : printed far too much.
  - `INTERNAL_ERROR` : the judge itself failed (never blamed on the user; the submission is retried or flagged).
- **Status flow:** `QUEUED` then `RUNNING` then a final verdict. A final verdict never changes again (unless an admin re-judges).
- **Contest:** a time window with a set of problems and its own leaderboard. Scoring: more problems solved ranks higher; ties are broken by lower total penalty time.
- **Sandbox:** a throw-away, locked-down container that runs one submission and is destroyed afterwards.
- **Time convention:** store UTC; display and schedule contests in IST.

4) Feature Requirements

M1: Identity & Access

- Register and log in (bcrypt-hashed passwords), **JWT in HTTP-only cookies**, logout, RBAC middleware.
- Login rate limit and lockout after repeated failures, kept in **Redis** (atomic counters with a TTL).
- Frontend: protected routes and a clean logout that clears cached data.

M2: Problems

- Problem setters create problems: title, statement (Markdown), time and memory limits, sample and hidden test cases.
- **Test case files are stored in MinIO** (object storage), not in the database. The database stores only their keys.
- Browse problems with **search** (Postgres full-text search) and filters (difficulty, tag), with **cursor pagination**.

- Frontend: problem list, and a problem page with the statement and a code editor (**Monaco**).
- Problem details are cached in Redis (cache-aside), and invalidated when the problem is edited.

M3: Submit and Judge (the core)

1. The user submits code. The API validates it (size limit, language), stores it, creates a submission as `QUEUED`, puts a job on the **queue**, and returns `202 Accepted` immediately with a submission id.
2. A **judge worker** takes the job and, for each test case, runs the code inside a **sandbox container** with the limits from Part 2, then compares the output.
3. The worker stores the verdict and per-test results, then publishes an event.
4. The browser, which is **already subscribed to live updates** for this submission, shows `QUEUED` then `RUNNING` then the verdict without refreshing.

Details:

- **Output comparison:** compare after trimming trailing whitespace on each line and trailing blank lines. Document your rule.
- **Stop early:** stop at the first failing test (and record which test, but never reveal hidden test content; only reveal the index).
- **Idempotency:** if a job is delivered or retried twice, the submission must end with **one** verdict and **no duplicate** leaderboard credit.
- **Retries:** an `INTERNAL_ERROR` (for example, Docker was unavailable) is retried with backoff up to a limit; a user's code failing is **never** retried.
- **Stuck-job recovery:** a submission stuck in `RUNNING` beyond a threshold (for example the worker was killed) is detected by a scheduled job and re-queued once, then marked `INTERNAL_ERROR`.
- **Submit protection:** a per-user submission **rate limit** (for example 5 per minute) in Redis, and a guard that rejects an identical submission sent twice within a few seconds (double-click).

M4: Live Updates

- The server pushes status changes to the browser using **Server-Sent Events (SSE)**. Verdict events travel from the worker to the API through **Redis pub/sub**, so it works with more than one API instance.
- The submission page shows live status. If the connection drops, it reconnects and fetches the current state (no missed verdict).
- A "my submissions" table with live row updates and **cursor pagination**.

M5: Leaderboard

- A global leaderboard (problems solved, then total penalty) kept in a **Redis sorted set**, updated when a first `ACCEPTED` verdict is recorded for a user and problem.

- Re-solving the same problem never adds points twice (idempotent).
- The leaderboard can be rebuilt from Postgres at any time (a rebuild job), and you must show that the rebuilt copy matches the live one.
- Frontend: a leaderboard page that updates live.

M6: Contests (Tier 2)

- Admin creates a contest: problems, start and end time (IST).
- Scheduled **BullMQ jobs** open the contest at the start time, close it at the end time (submissions after the end are rejected), and **freeze** the leaderboard for the last N minutes.
- A per-contest leaderboard with the scoring rule in Part 1.

M7: Events & Notifications (Tier 2)

- Each final verdict is published as an event to a **Kafka** topic (`submission.judged`).
- Independent consumers read it: one **updates the leaderboard** and one **sends an email** (to a stub gateway) when a user solves a problem for the first time. Each consumer is **idempotent** and has a retry cap and a dead-letter topic.

M8: Admin & Operations

- Admin page: queue depth, running submissions, verdict counts, average judge time, failed jobs, and buttons to **re-judge** and **kill** a submission.
- `/health/live` and `/health/extensive` (checks Postgres, Redis, MinIO, Kafka, and that Docker is reachable).
- Structured JSON logs with a **submission id** carried through the API, the queue job and the worker so one submission can be traced.
- A **cleanup cron** removes leftover containers, temp files and old per-test outputs.
- Metrics endpoint (`/metrics`): queue length, verdict counts, judge duration, container start failures.

M9: Frontend

- React + Vite + TypeScript. **TanStack Query** with a **query-key factory** for server data; **Zustand** only for small client state (editor preferences, for example). Do not copy server data into a global store.
- Monaco editor with language mode, a sample-input "Run" box (runs only against the sample, and still sandboxed), and a result panel.
- Axios with a **401 refresh queue**, loading / empty / error states everywhere, keyboard accessibility, and an English / Telugu toggle for the interface labels (stretch).

PART 2: SAFETY RULES & ARCHITECTURE

1) Safety Rules for Running Untrusted Code (Non-Negotiable)

You are building a system that executes arbitrary code that strangers write. If this is done carelessly, a user can read your files, delete your data or take over your computer. **These rules are mandatory and will be checked in your review.**

1. **Local only.** The judge runs **only on your own machine**. Do not deploy it, expose it with a tunnel, or let anyone else submit code to it. Any exposure needs written sign-off from your mentor first.
2. **Every run happens in a fresh container, and the container is destroyed afterwards.** Never run submitted code directly on your host or in the worker's own container.
3. **Every sandbox container must be started with:**
   - `--network none` (no network at all)
   - a **memory limit** (for example `--memory=128m`, with swap disabled via `--memory-swap` equal to the memory limit)
   - a **CPU limit** (for example `--cpus=0.5`)
   - a **process limit** (for example `--pids-limit=64`, to stop fork bombs)
   - `--read-only` root filesystem, with a small writable `tmpfs` for scratch (size-limited)
   - `--cap-drop ALL` and `--security-opt no-new-privileges`
   - a **non-root user**
   - a hard **wall-clock timeout** enforced by *your worker* (kill the container if it runs too long), in addition to any limit inside the container
4. **Cap the output.** Read at most a fixed number of bytes from the container's output (for example 64 KB), then kill it. Never load unbounded output into memory.
5. **Never mount your host source code, home folder, `.env` files or the Docker socket into a sandbox container.** The sandbox gets only the submitted code and one test input.
6. **Do not put the Docker socket inside another container.** A container that can talk to the Docker socket is effectively root on your machine. Run the judge worker as a normal process **on the host** (outside Compose) and let it start sandbox containers through the Docker CLI or API, and document why.
7. **Use a minimal, pinned image** for the sandbox (for example a slim Node image pinned to a specific tag), pre-pulled so runs never need the network.
8. **Never trust anything the submitted code prints.** Treat output as untrusted data: it is never rendered as HTML and never executed.
9. **Hidden test data never leaves the server.** Do not return hidden inputs or expected outputs in any API response, log or error message.

10. **You must write the "attack suite" (see AC-SAFE-01 to AC-SAFE-08)**: a folder of hostile submissions that prove each protection works, run automatically as part of your tests.

2) Services (Docker Compose, plus the host-run worker)

Service	Purpose
`postgres`	Users, problems, submissions, contests, results
`redis`	Rate limits, caches, leaderboard, pub/sub, BullMQ queue
`minio`	Test case files, archived source, archived outputs
`kafka` (+ UI)	Event stream for verdicts (Tier 2)
`api`	Express REST API and the SSE endpoint
`cron`	BullMQ schedulers (contest open and close, stuck-job recovery, cleanup, rollups)
`notifier`, `leaderboard-consumer`	Kafka consumers (Tier 2)
`stub-mail`	A tiny HTTP service you build that pretends to send email, with switchable failures
`prometheus`, `grafana` (optional)	Metrics and dashboards
`frontend`	The React app
judge worker	**Runs on the host, outside Compose** (see Safety Rule 6). It talks to Redis and Postgres exposed by Compose, and to the local Docker engine.

3) How a submission flows

```
Browser --POST /submissions--> API  --(validate, store, rate-limit)--> Postgres: QUEUED
                                    \--> BullMQ queue (Redis) --> returns 202 + id
Browser <--SSE (Redis pub/sub)-- API <--publish-- Judge worker
                                                    |  for each test case (fetch from MinIO):
                                                    |    start sandbox container -> run -> compare
                                                    |  store verdict -> Postgres
                                                    |  publish live status -> Redis pub/sub
                                                    '-> publish event -> Kafka (Tier 2)
Kafka --> leaderboard consumer (Redis sorted set) and notifier (stub-mail)
```

4) Pattern Cards (study these; you must explain each one in your own words)

Card 1: Queue and worker. The API never runs code itself. It records the work and puts a job on a queue; a worker takes jobs at its own pace. This keeps the API fast and lets you control how many runs happen at once. Learn BullMQ: concurrency, rate limiter, retries with backoff, lock duration, and why a worker needs an `error` handler.

Card 2: Idempotent handlers. Jobs are delivered at least once and can be retried, so running the same job twice must be safe. Check state first; update with a condition (`UPDATE ... WHERE status IN ('QUEUED','RUNNING')`); give each leaderboard credit a unique key.

Card 3: Your fault vs the system's fault. A wrong answer is the user's fault and is final. A Docker failure is the system's fault and is retried. Never mix the two.

Card 4: Resource limits and timeouts. Limits inside the container protect the host; a timeout enforced *outside* protects the worker. You need both, and a "kill, then clean up" path that always runs (`finally`).

Card 5: Pub/sub for live updates. Redis pub/sub delivers an event to every subscribed API instance, which forwards it to its connected browsers over SSE. Pub/sub does not store messages, so the page must also fetch current state on connect.

Card 6: Sorted sets. `ZADD` , `ZINCRBY` , `ZREVRANGE` , `ZRANK` give you a leaderboard that stays sorted for free. Learn how to encode "problems solved, then lower penalty" into one score.

Card 7: Fixed-window counters. `INCR` the key; if the result is 1, `EXPIRE` it. Understand the tiny gap if the process dies between the two calls and how to close it (a Lua script, or `SET key 0 EX ttl NX` then `INCR`).

Card 8: Cache-aside. Read the cache; on a miss read the database and fill the cache with a TTL; on a change, delete the key.

Card 9: Presigned URLs and object storage. Test case files live in MinIO. Learn buckets, keys and presigned URLs, and why large files do not belong in the database.

Card 10: Scheduled jobs. A scheduler gives you a time, not "only one instance runs it". Protect each tick with a unique run claim, and make jobs safe to run twice.

Card 11: Recovering stuck work. A worker can be killed at any moment, leaving a submission in `RUNNING` forever. A scheduled sweeper finds work that has been running too long and re-queues it once, then fails it.

Card 12: Fail open vs fail closed. If Redis is down, should the submit rate limiter allow or deny? Decide deliberately and write down why. (Hint: it is not the same answer as for the sandbox limits, which must always fail closed.)

Card 13: Server state vs client state. Server data belongs in TanStack Query's cache; do not copy it into a global store. Use query-key factories and targeted invalidation.

PART 3: EXPECTED OUTCOMES & ACCEPTANCE CRITERIA

Format: Given / When / Then. Reference each AC ID in `docs/acceptance-map.md` (which test or demo step proves it). Tier labels are explained in Part 5.

1) Expected Outcomes

- A reviewer can run the platform with Compose plus the host worker and watch a submission go from `QUEUED` to a verdict live.
- A hostile submission can never harm the host, leak data, hang the judge or leave containers behind.
- You can **explain every service, limit and pattern**, and why it exists.

2) Acceptance Criteria

Identity (Tier 1)

- **AC-ID-01**: Given a login, when inspected, then tokens are in HTTP-only cookies and passwords are stored only as bcrypt hashes.
- **AC-ID-02**: Given repeated failed logins, when the limit is reached, then the account is locked out for a window, and the response does not reveal whether the user exists.
- **AC-ID-03**: Given a role without permission, when it calls a protected route, then the response is 403; routes are denied by default unless whitelisted.
- **AC-ID-04**: Given Redis is stopped, when a user logs in normally, then the login rate limiter fails open (and the event is logged) and login still works.

Problems (Tier 1)

- **AC-PR-01**: Given a new problem with hidden tests, when created, then test files are in MinIO and the database holds only their keys.
- **AC-PR-02**: Given a problem API response, when inspected, then **no hidden test input or output** appears anywhere.
- **AC-PR-03**: Given a search, when run, then results are ranked and filters combine with AND.
- **AC-PR-04 (Pagination)**: Given more problems than a page, when paging with a cursor, then there are no repeated or skipped rows even as new problems are added.
- **AC-PR-05**: Given an edited problem, when saved, then its cache entry is invalidated.

Submit & Judge (Tier 1)

- **AC-SJ-01**: Given a submission, when posted, then the API returns `202` quickly with an id and does not wait for the judge.

- **AC-SJ-02**: Given a correct solution, when judged, then the verdict is `ACCEPTED` and every test result is stored.
- **AC-SJ-03**: Given a wrong solution, when judged, then the verdict is `WRONG_ANSWER` and only the **index** of the failing test is revealed.
- **AC-SJ-04**: Given a solution with a syntax error, when judged, then the verdict is `COMPILATION_ERROR` with a safe, truncated message.
- **AC-SJ-05**: Given a solution that throws, when judged, then the verdict is `RUNTIME_ERROR`.
- **AC-SJ-06 (Idempotency)**: Given the same job delivered twice, when both run, then there is **one** final verdict and no duplicate leaderboard credit.
- **AC-SJ-07**: Given Docker is unavailable, when a job runs, then it is `INTERNAL_ERROR`, retried with backoff up to a limit, and **never blamed on the user**.
- **AC-SJ-08**: Given the worker is killed mid-run, when the sweeper runs, then the stuck submission is re-queued once and, if it fails again, marked `INTERNAL_ERROR`.
- **AC-SJ-09**: Given the output-comparison rule, when two outputs differ only by trailing whitespace, then they are treated as equal (and the rule is documented and tested).
- **AC-SJ-10 (Rate limit)**: Given a user submits more than the limit in a minute, when the next submission arrives, then it is rejected with `429` and a retry-after value.
- **AC-SJ-11 (Double click)**: Given two identical submissions within seconds, when received, then only one is queued.

Sandbox & Attack Suite (Tier 1, mandatory)

- **AC-SAFE-01 (Infinite loop)**: Given `while(true){}`, when judged, then the verdict is `TIME_LIMIT_EXCEEDED`, the container is killed and removed, and the worker stays healthy.
- **AC-SAFE-02 (Memory bomb)**: Given code that allocates memory until it fails, when judged, then the verdict is `MEMORY_LIMIT_EXCEEDED` and the host is unaffected.
- **AC-SAFE-03 (Fork bomb)**: Given code that spawns processes endlessly, when judged, then the process limit stops it and the host is unaffected.
- **AC-SAFE-04 (Network)**: Given code that tries to make an outbound request, when judged, then it fails because the container has no network.
- **AC-SAFE-05 (Filesystem)**: Given code that tries to read host files, `.env` files or write outside its scratch space, when judged, then it fails and nothing outside the sandbox is readable or changed.
- **AC-SAFE-06 (Output flood)**: Given code that prints gigabytes, when judged, then reading stops at the cap, the verdict is `OUTPUT_LIMIT_EXCEEDED` and memory stays flat.
- **AC-SAFE-07 (Privileges)**: Given a submission, when its container is inspected, then it runs as non-root with all capabilities dropped and `no-new-privileges` set.
- **AC-SAFE-08 (Cleanup)**: Given 100 mixed submissions, when all are done, then there are **zero** leftover containers, and temporary files are removed.

- **AC-SAFE-09**: Given the repository, when inspected, then the Docker socket is not mounted into any container, and the host-run worker design is documented.

Live Updates (Tier 1)

- **AC-LV-01**: Given a submission page, when the verdict is produced, then the browser updates over SSE without a refresh.
- **AC-LV-02**: Given two API instances running, when a verdict is produced by the worker, then a browser connected to **either** instance receives it (pub/sub).
- **AC-LV-03**: Given the connection drops and returns, when the page reconnects, then it fetches the current state and shows the correct verdict (no missed update).
- **AC-LV-04**: Given a final verdict, when received, then the UI stops listening (no leaked connections).

Leaderboard (Tier 1)

- **AC-LB-01**: Given a first `ACCEPTED` for a user and problem, when recorded, then the leaderboard score updates.
- **AC-LB-02**: Given the same user re-solving the same problem, when judged, then the score does **not** change.
- **AC-LB-03**: Given the Redis leaderboard is wiped, when the rebuild job runs, then it matches what Postgres says (shown with a comparison).
- **AC-LB-04**: Given ties, when ranked, then the lower total penalty ranks higher.

Contests, Events & Operations (Tier 2)

- **AC-CT-01**: Given a contest start time (IST), when it arrives, then it opens automatically; at the end it closes and late submissions are rejected.
- **AC-CT-02**: Given the freeze window, when it begins, then the public leaderboard stops changing, while the admin view still updates.
- **AC-CT-03**: Given two cron containers, when a tick fires, then exactly one performs the work (unique run claim).
- **AC-EV-01**: Given a final verdict, when published to Kafka, then the leaderboard consumer and the notifier each process it exactly once in effect, even if it is delivered twice.
- **AC-EV-02**: Given a message that keeps failing, when it reaches the retry cap, then it goes to the dead-letter topic with its error.
- **AC-EV-03**: Given the stub mail service fails or times out, when sending, then the notifier retries with backoff and never blocks the leaderboard consumer.
- **AC-OP-01**: Given a submission, when traced, then the same submission id appears in the API log, the queue job and the worker log.

- **AC-OP-02**: Given `/metrics`, when scraped, then it exposes queue length, verdict counts, judge duration and container-start failures.
- **AC-OP-03**: Given `SIGTERM` during a run, when the worker shuts down, then the current container is killed and removed, the job is returned to the queue, and the process exits cleanly.
- **AC-OP-04**: Given `/health/extensive`, when a dependency (including Docker) is stopped, then it reports that dependency as down.

Frontend (Tier 1 and 2)

- **AC-FE-01**: Given any list, when loading, empty or failing, then a skeleton, an empty state or an error with Retry is shown.
- **AC-FE-02**: Given ten parallel requests with an expired access token, when they fire, then the refresh endpoint is called **once** and all ten succeed.
- **AC-FE-03**: Given two quick searches, when responses arrive out of order, then the UI shows the latest, not a stale one.
- **AC-FE-04**: Given a code editor, when the user presses Run on a sample, then the result appears in the result panel and runs through the same sandbox.
- **AC-FE-05**: Given any interactive element, when used by keyboard, then it is reachable with a visible focus state.

3) Edge Cases to Handle

- **EC-01**: Two workers pick the same job at once (BullMQ lock expiry).
- **EC-02**: The worker is killed after the verdict is saved but before the event is published.
- **EC-03**: A submission arrives one second after a contest ends.
- **EC-04**: Code that never ends **and** ignores signals.
- **EC-05**: Code that prints invalid UTF-8 or null bytes.
- **EC-06**: A test case file is missing from MinIO (judge failure, not a user failure).
- **EC-07**: Very large source code (over the size limit).
- **EC-08**: A problem is edited while submissions for it are queued (decide which test version they use, and document it).
- **EC-09**: SSE client connects after the verdict is already final.
- **EC-10**: The leaderboard credit is given, then an admin re-judges the submission to a different verdict (decide and document).

4) Final Validation Checklist (Go / No-Go)

1. `docker compose up` plus the host worker start the platform from a clean clone.

2. A user submits code and watches the verdict update live.
3. **Every attack in the suite is contained**, and no containers are left behind.
4. Duplicate delivery never causes a duplicate verdict or leaderboard credit.
5. Killing the worker mid-run never leaves a submission stuck forever.
6. The leaderboard can be rebuilt and matches.
7. A submission can be traced by id across services; metrics work.
8. The safety rules are followed and documented.
9. Tests pass; no secrets committed; architecture and safety reports written.

PART 4: TECH STACK REQUIREMENTS

Excludes: cloud deployment, Kubernetes.

1) Core

- **Language:** TypeScript (strict). Node.js LTS (v20). pnpm (or npm; be consistent).
- **Repository:** monorepo with `apps/api`, `apps/judge-worker`, `apps/cron`, `apps/stub-mail`, `apps/frontend`, and `packages/shared` (types, schema, utilities).
- **Local environment:** Docker Compose for the services in Part 2; Docker Engine for sandboxes.

2) Backend

- **API:** Express, REST, JSON. Validation with `zod`. One error shape: `{ code, message }`.
- **Database:** PostgreSQL with **Drizzle ORM** (or Prisma) and migrations. Pagination and search queries must be **plain SQL** you can read and explain.
- **Redis:** `ioredis`. Use a **separate connection for BullMQ**. Namespace keys with an app prefix.
- **Queue: BullMQ** for the judge queue (concurrency, limiter, retries, backoff, lock duration) and scheduled jobs (schedulers with `tz: 'Asia/Kolkata'`, run claims).
- **Events (Tier 2):** `kafkajs` with manual offset commits, a retry cap and a dead-letter topic.
- **Object storage:** MinIO with the S3-compatible SDK.
- **Sandbox control:** the Docker CLI via `child_process` (`execFile`, never `exec` with strings) or the `dockerode` library. **Never** build a shell command by concatenating user input.
- **Live updates:** Server-Sent Events from Express, Redis pub/sub between processes.
- **Auth and security:** `bcrypt`, `jsonwebtoken`, `helmet`, `express-rate-limit` with a Redis store.
- **Observability:** `pino` for JSON logs, `prom-client` for metrics.

- **Fake data:** `@faker-js/faker` with a fixed seed.

3) Frontend

- React + Vite + TypeScript, Tailwind CSS (shadcn/ui optional), recharts.
- **Monaco editor** (`@monaco-editor/react`).
- **TanStack Query** (with a key factory) for server state; **Zustand** for small client state; TanStack Table for lists.
- Axios with interceptors; react-router; react-hook-form with zod; an SSE hook with reconnect.

4) Testing & QA

- **Unit and integration:** Vitest, with a real test Postgres, Redis and (for Tier 2) Kafka.
- **The attack suite:** a folder of hostile submissions (infinite loop, memory bomb, fork bomb, network call, file read, output flood, privilege checks) run by an automated test against the real sandbox.
- **Concurrency tests:** parallel submissions; duplicate job delivery; worker kill and sweeper recovery.
- **Required:** a test for each Tier 1 acceptance criterion. **At least 35 meaningful tests.**
- **Stretch:** Playwright for one end-to-end journey; GitHub Actions for lint, type-check and unit tests (the attack suite stays local).

5) Quality & Security Standards

- ESLint and Prettier; Conventional Commits.
- All API inputs validated; no secrets in the repo; `.env.example` only.
- Never log source code of other users, hidden tests, or tokens.

PART 5: SUBMISSION DETAILS

Note: No cloud deployment is required. The judge must **never** be exposed publicly.

1) How This Project Is Scored (Tiers)

This is deliberately ambitious for 8 weeks. You are **not** expected to finish everything. Work in tiers and **finish each tier properly** before starting the next.

- **Tier 1 (required):** M1 Identity, M2 Problems, M3 Submit and Judge, M4 Live Updates, M5 Leaderboard, the full attack suite, the host-run worker, and the core documentation.
- **Tier 2 (expected if on track):** M6 Contests, M7 Kafka events and notifier, M8 Admin and Operations, metrics, English / Telugu labels.

- **Tier 3 (stretch):** a second language (Python) with its own image and limits, a similarity (plagiarism) check, partial scoring, a custom "checker" for problems with many valid answers, Playwright, GitHub Actions, a Grafana dashboard.

A fully working, well-tested and **safe** Tier 1 is better than a half-working everything. Your mentor will help you decide, week by week, what to cut.

2) What You Hand In

1. A **private GitHub repository**, invited to our organization.
2. The full codebase, with one documented command to start Compose and one to start the host worker.
3. A `/docs` folder (Section 4 below).
4. A **demo video (8-10 minutes)**: submit correct and wrong solutions with live verdicts, then **run the attack suite live**, then kill the worker mid-run and show recovery, then show the leaderboard rebuild.
5. Test results.

3) Codebase Requirements

- Prettier / ESLint; Conventional Commits (`type(scope): summary`).
- **Commit at least three times a week**; plan your own weeks with GitHub issues and link commits to them.
- One `README` per app explaining what it does and how to run it.

4) Documentation You Must Include (in `/docs`)

1. `README.md` (root): what it is, how to run it, how to test it, a service map.
2. `architecture.md` : a diagram of all services, the full submission flow, and a short "**why**" for each major choice.
3. `safety.md` : every safety rule, how you implemented it, and how the attack suite proves it. Include the **Docker command line** you use for a sandbox, with each flag explained.
4. `decisions.md` : at least 8 short decision records ("I chose X over Y because...").
5. `pattern-notes.md` : for each Pattern Card, a paragraph in **your own words** and where you used it.
6. `failure-report.md` : each failure experiment (kill the worker, stop Redis, stop Docker, flood the queue), what you expected, what happened, what you changed.
7. `runbook.md` : how to add a problem, how to re-judge, how to recover a stuck submission, how to clean up containers, how to add a new language.
8. `acceptance-map.md` , `test-plan.md` , `changelog.md` , `known-issues.md` , `learning-notes.md` .

5) Weekly Plan and Check-in Demos

Week	Focus	New things you will learn	Demo at the check-in
1	**Foundations and the sandbox.** Compose with Postgres, Redis and MinIO; monorepo; schema and migrations; API skeleton. Run **one** hardcoded program in a locked-down container from a script and capture the output. Primers (below).	Docker, container limits, `execFile`, Drizzle migrations	Run a script that executes a JS file in a sandbox, then show an infinite loop being killed at the timeout.
2	**Identity and problems.** M1 and M2: auth, RBAC, Redis rate limit and lockout; problems, test cases in MinIO, search and cursor pagination; React shell and the problem pages with Monaco.	JWT cookies, Redis counters, full-text search, MinIO	Log in, create a problem with hidden tests, search it, view it in the editor.
3	**The judge core.** M3: submit, BullMQ queue, host-run worker, per-test execution, output comparison, verdicts, idempotent updates.	BullMQ, queues and workers, idempotency, exit codes	Submit correct, wrong, syntax-error and crashing code and see each verdict.
4	**The attack suite.** All Tier 1 sandbox criteria, output caps, cleanup, `INTERNAL_ERROR` retries, stuck-job sweeper.	Resource limits, timeouts, failure classification, cron sweepers	Run the attack suite live; kill the worker mid-run and watch the sweeper recover.
5	**Live updates.** M4: SSE, Redis pub/sub, reconnect and catch-up; live status UI; a "my submissions" table.	SSE, pub/sub, reconnection logic	Open two browser tabs on two API instances; both update live.
6	**Leaderboard and rate limits.** M5: sorted-set leaderboard, idempotent credit, rebuild job; submit	Redis sorted sets, atomic updates	Duplicate-deliver a job and show no double credit; wipe Redis and rebuild.

	rate limit and double-click guard; leaderboard UI.		
7	**Contests and events (Tier 2).** M6 contests with scheduled open, close and freeze; M7 Kafka verdict events with two idempotent consumers and a dead-letter topic.	BullMQ schedulers, Kafka consumers, offsets, DLQ	Run a contest that opens and closes on schedule; replay a Kafka message and show no duplicate.
8	**Operations and wrap-up.** M8: admin page, metrics, health, graceful shutdown, structured logs with submission ids; documentation; demo video.	Metrics, tracing by id, graceful shutdown	The final 20-minute walkthrough and the failure demo.

6) Week 1 Concept Check

Before the end of week 1, write a short paragraph in your own words on each (`docs/learning-notes.md`):

1. What a Docker container is, and how it differs from a virtual machine.
2. What `--network none`, `--memory`, `--pids-limit` and `--read-only` do.
3. Why running untrusted code on your host is dangerous, and why mounting the Docker socket is as bad as giving root.
4. What a message queue is and why a judge uses one.
5. What Redis is, its data types, why TTLs matter, and what `INCR` and sorted sets do.
6. What a database transaction is, and what **idempotent** means with an example.
7. `fetch` vs Axios, `useEffect` (dependency array and cleanup), Promises and `async/await`.
8. Hashing vs encryption, how to store passwords, and what is inside a JWT.
9. What Server-Sent Events are and how they differ from polling and WebSockets.

7) Evaluation Rubric

- **Safety and correctness (35%):** the attack suite passes, no leftover containers, no leaks of hidden data, no duplicate verdicts or credits.
- **Completeness (25%):** the tiers you delivered, working end to end.
- **Understanding (20%):** you can explain every flag, limit and pattern; `safety.md`, `pattern-notes.md` and your live explanations.
- **Quality (10%):** tests, code organization, commits and issues.

- **Documentation and demo (10%):** clear README, architecture, runbook and demo.

Beyond the rubric we also look at: weekly improvement, how you handle being stuck, and how honestly you report problems.

8) How to Work (and how to ask for help)

- Read the official docs first (Docker, BullMQ, Redis, PostgreSQL, MinIO, Server-Sent Events). Take notes.
- Build the smallest thing that works, then improve it. Commit often.
- Stuck for more than a couple of hours? Write down what you tried and ask. That is expected, not a failure.
- **Any safety doubt: stop and ask first.**
- Using AI tools is fine for explanations, but **you must be able to explain and modify every line you submit**, especially the sandbox command. We will ask.

9) Legal, Privacy & Security Notes

- All data is fabricated. No real personal data anywhere: repository, logs or demo.
- `.env.example` only; never commit keys.
- The judge is for local use only. Do not expose it, and do not run it on a shared or cloud machine.

10) Submission Checklist

- ◯ Private GitHub repo invited to our organization
- ◯ Platform starts from a clean clone (Compose plus the host worker, both documented)
- ◯ Tier 1 complete and tested (state clearly which Tier 2 and 3 items you reached)
- ◯ The full attack suite passes and leaves no containers behind
- ◯ Duplicate-delivery, worker-kill and leaderboard-rebuild tests pass
- ◯ `/docs` includes everything in Section 4, including `safety.md`
- ◯ Demo video (8-10 minutes) with the live attack suite and a failure experiment
- ◯ Tests passing (35+); no secrets committed; `.env.example` included
- ◯ The Docker socket is not mounted into any container

If anything is unclear, ask **before** building. Keep all assumptions in `docs/known-issues.md` or `docs/changelog.md`. This is a hard project, and the hardest part is keeping it safe. Treat every problem you hit as the actual lesson. Good luck.